CODE :-

```
#include <stdio.h>

int main()
{
   int n, m;
   int i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   int allocation[n][m];
   int max[n][m];
   int need[n][m];
   int available[m];
   int work[m];
   int finish[n];
   int safeSequence[n];

   // Input Allocation Matrix
   printf("\nEnter Allocation Matrix:\n");
   for (i = 0; i < n; i++)
   {
      printf("P%d: ", i);
      for (j = 0; j < m; j++)
      {
         scanf("%d", &allocation[i][j]);
      }
   }

   // Input Maximum Matrix
   printf("\nEnter Maximum Matrix:\n");
   for (i = 0; i < n; i++)
   {
      printf("P%d: ", i);
      for (j = 0; j < m; j++)
      {
         scanf("%d", &max[i][j]);
      }
   }

   // Input Available Resources
   printf("\nEnter Available Resources:\n");
   for (j = 0; j < m; j++)
   {
      scanf("%d", &available[j]);
```

```
}

// Calculate Need Matrix
for (i = 0; i < n; i++)
{
   for (j = 0; j < m; j++)
   {
      need[i][j] = max[i][j] - allocation[i][j];
   }
}

// Display Need Matrix
printf("\nNeed Matrix:\n");
for (i = 0; i < n; i++)
{
   printf("P%d: ", i);
   for (j = 0; j < m; j++)
   {
      printf("%d ", need[i][j]);
   }
   printf("\n");
}

// Initialize Work and Finish
for (j = 0; j < m; j++)
   work[j] = available[j];

for (i = 0; i < n; i++)
   finish[i] = 0;

// Find Safe Sequence
int count = 0;

while (count < n)
{
   int found = 0;

   for (i = 0; i < n; i++)
   {
      if (finish[i] == 0)
      {
         int possible = 1;

         // Check Need <= Work
         for (j = 0; j < m; j++)
         {
            if (need[i][j] > work[j])
            {
               possible = 0;
               break;
            }
```

```
            }

            if (possible)
            {
               // Process can execute
               for (k = 0; k < m; k++)
               {
                  work[k] += allocation[i][k];
               }

               safeSequence[count] = i;
               count++;
               finish[i] = 1;
               found = 1;
            }
         }
      }

      if (found == 0)
      {
         break;
      }
   }

   // Check whether system is safe
   if (count == n)
   {
      printf("\nSystem is in a SAFE state.\n");

      printf("Safe Sequence: ");
      for (i = 0; i < n; i++)
      {
         printf("P%d", safeSequence[i]);

         if (i != n - 1)
            printf(" -> ");
      }
      printf("\n");
   }
   else
   {
      printf("\nSystem is in an UNSAFE state.\n");
      printf("Deadlock may occur.\n");
   }

   return 0;
}
```

INPUT :-

Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

Enter Maximum Matrix:
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3

Enter Available Resources:
3 3 2

OUTPUT :-

Need Matrix:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

System is in a SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2